printf的位置不同导致递归过程不同

- 以前序递归为例：

```
void preorder(TreeNode *root)
{
  if (root == NULL)
      return;
    printf("%d ", root→data);
    preorder(root→left);
    preorder(root→right);
}
```

假如二叉树为：

第一层：1

第二层：2、3

第三层：4、5、6、7

- 根节点为1，判断存在，打印1，按顺序执行preorder(2),preorder(3)
- 先执行preorder(2),判断节点存在，打印2，按顺序执行preorder(4),preorder(5)
- 到preorder(4)时打印4，然后发现4的左孩子与右孩子均为空节点，返回preorder(5),打印5后发现5的左孩子与右孩子均为空节点，此时preorder(2)执行完毕，开始执行preorder(3)
- preorder(3)时先打印3，然后按顺序执行preorder(6)与preorder(7),与上一步执行方法相同
- 最后打印得：1，2，4，5，3，6，7

计算树的深度函数的后两个形参不用指针的原因

- 当我输入实参时，相当于把实参复制了一份交给函数计算，最后的return会返回计算的值，所以不用指针直接修改内存

递归与循环的不同

- 循环时重复执行同一个任务，同一个过程不断重复
- 递归是一个拆分的过程，把一个大问题不断分解为几个小问题再按顺序做，类似复合函数